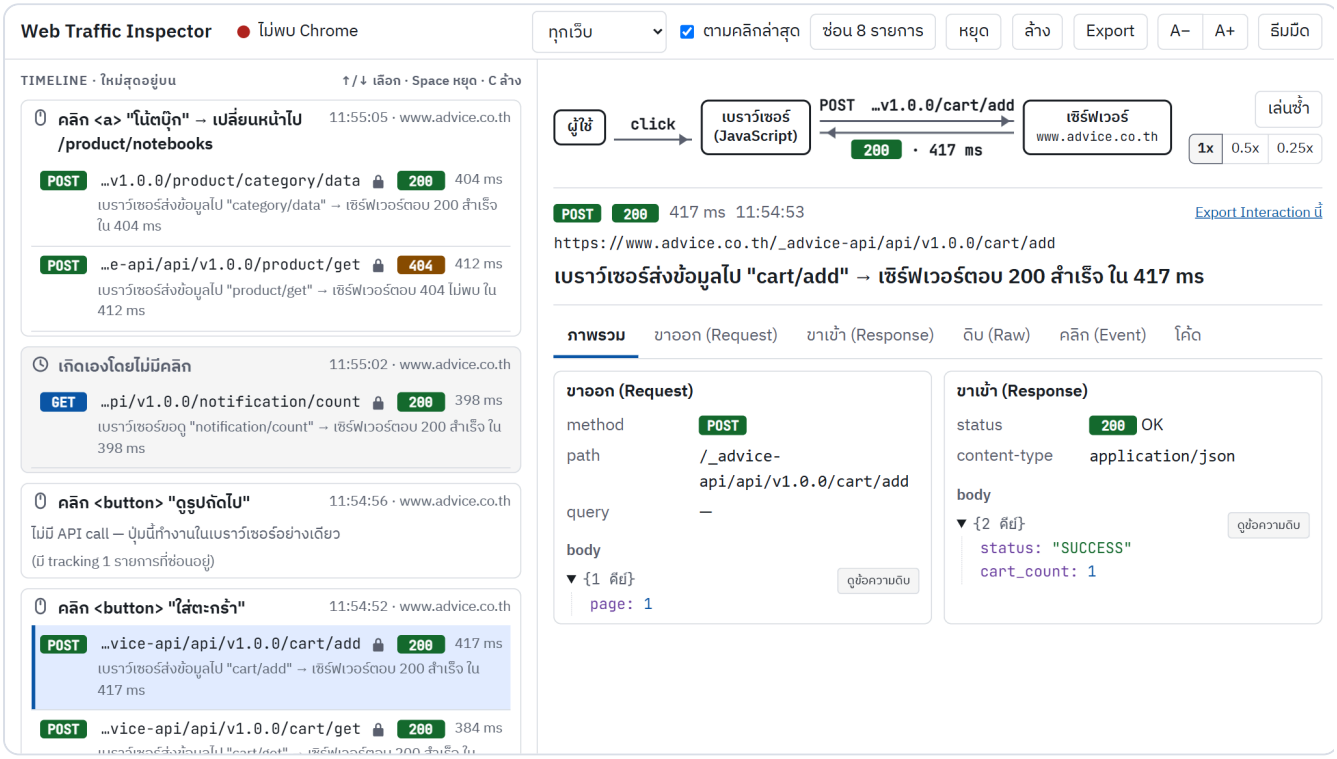
คู่มือการใช้งาน · เวอร์ชัน 1

Web Traffic Inspector

ดูว่าคลิกหนึ่งครั้งบนเว็บ ทำให้เบราว์เซอร์ส่ง request อะไร และเซิร์ฟเวอร์ตอบอะไรกลับมา



สำหรับผู้สอนวิชาเว็บและเครือข่าย · ใช้ได้ทั้ง Windows และ macOS · ภาพหน้าจอในคู่มือนี้ถ่ายจากโหมด `--demo` (ข้อมูลตัวอย่าง)

สารบัญ

1. Inspector คืออะไร
2. ติดตั้ง (ครั้งเดียว)
3. เริ่มใช้งานใน 3 ขั้น
4. รู้จักหน้าจอ
5. แท็บใน Detail
6. แถบเครื่องมือและปุ่มลัด
7. Preset และ Exchange ที่ถูกซ่อน
8. Export และเปิดไฟล์ Recording
9. ตัวเลือก command line
10. ความปลอดภัยและข้อมูลลับ
11. ข้อจำกัดที่ควรรู้
12. เคล็ดลับการสาธิตในห้อง
13. แก้ปัญหาที่พบบ่อย

1. Inspector คืออะไร

Web Traffic Inspector เป็นเครื่องมือสอนที่ต่อเข้ากับ Chrome ของผู้สอนผ่าน **Chrome DevTools Protocol (CDP)** แบบดูอย่างเดียว แล้วแสดงสิ่งที่เกิดขึ้นเมื่อผู้ใช้คลิกบนเว็บ เป็นลำดับ **คลิก** → **request** → **response** ที่หน้า `http://127.0.0.1:8000`

- **ดูอย่างเดียว**: ไม่แก้ ไม่บล็อก และไม่ส่ง request ซ้ำไปยังเว็บ
- **ซ่อนค่าลับก่อนเก็บเสมอ**: cookie, token, password, รหัสติดตามผู้ใช้ ฯลฯ แสดงเป็น `••••` และปิดไม่ได้
- **ไม่เขียนลงดิสก์เอง**: ไฟล์จะเกิดขึ้นก็ต่อเมื่อกดปุ่ม Export เท่านั้น

ศัพท์ที่ใช้ในคู่มือ

คำ	ความหมาย
Exchange	request หนึ่งตัวคู่กับ response ของมัน (หนึ่งแถวใน Timeline)
Interaction	สาเหตุที่ทำให้เกิด Exchange: การคลิก, การเปิดหน้า หรือ "เกิดเองโดยไม่มีคลิก" (Background)
Timeline	คอลัมน์ซ้าย แสดง Interaction ใหม่สุดอยู่บน
Detail	คอลัมน์ขวา แสดงรายละเอียดของ Exchange ที่เลือก
Preset	ชุดกฎการกรอง ว่าจะแสดงเฉพาะเว็บไหน และซ่อนอะไร
Recording	ไฟล์ JSON ที่ได้จากปุ่ม Export ใช้เปิดดูย้อนหลังได้โดยไม่ต้องต่อ Chrome

> ใช้ดู traffic ของตัวเองเพื่อการเรียนรู้เท่านั้น อย่านำ API ของเว็บอื่นไปเรียกซ้ำหรือดึงข้อมูลจำนวนมาก

2. ติดตั้ง (ครั้งเดียว)

ต้องมี **Python 3.11 ขึ้นไป** และ **Google Chrome** แล้วเปิด Terminal ที่โฟลเดอร์ของโปรเจกต์

Windows (PowerShell)

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

ถ้าขึ้นว่า *running scripts is disabled* ตอนรัน `Activate.ps1` ให้รันคำสั่งนี้ครั้งเดียวแล้วลองใหม่

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

macOS (Terminal)

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

> ไม่ต้องรัน `playwright install` เพราะ Inspector ใช้ Chrome ที่ติดตั้งอยู่ในเครื่อง ไม่ได้ดาวน์โหลดเบราว์เซอร์เพิ่ม

ครั้งต่อไปที่จะใช้ ให้เปิด Terminal ที่โฟลเดอร์โปรเจกต์แล้วรันคำสั่ง activate (บรรทัดที่ 2 ด้านบน) ก่อนทุกครั้ง

3. เริ่มใช้งานใน 3 ขั้น

1. **เปิด Chrome โหมด debug** เลือกวิธีใดวิธีหนึ่ง
   - ดับเบิลคลิกสคริปต์: Windows ใช้ `scripts/start_chrome.bat`, macOS ใช้ `scripts/start_chrome.command` (ถ้า macOS ไม่ยอมเปิด ให้คลิกขวา → Open ในครั้งแรก หรือรัน `chmod +x scripts/start_chrome.command`)
   - หรือให้ Inspector เปิดให้เลย: `python -m inspector --launch-chrome` (รวมขั้นที่ 1 และ 2 ในคำสั่งเดียว)

   Chrome ตัวนี้ใช้**โปรไฟล์แยก** จึงไม่ปนกับบัญชีส่วนตัว และเปิดพร้อม Chrome ปกติได้

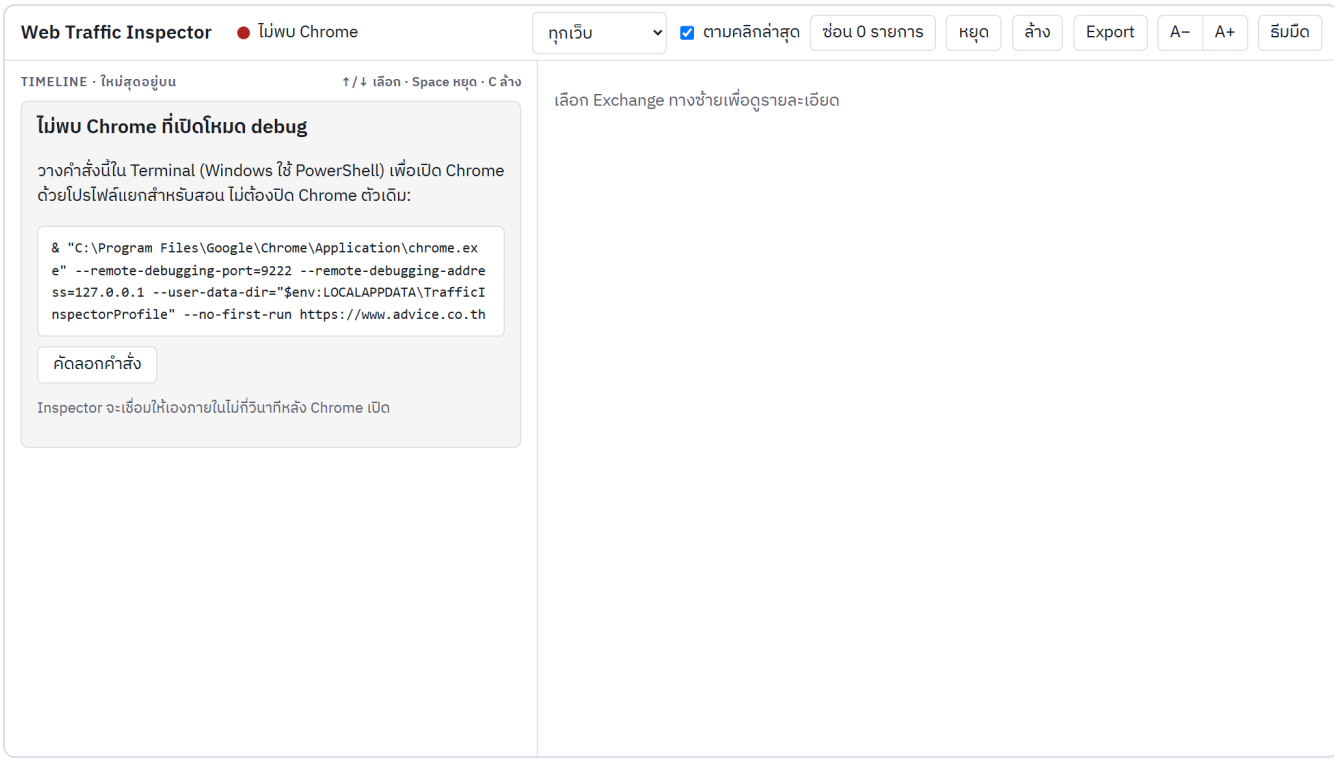
2. **รัน Inspector**

   ```
   python -m inspector                  # เชื่อม Chrome ที่เปิดไว้แล้ว
   python -m inspector --preset advice  # เริ่มด้วย preset ของ advice.co.th
   ```

   โปรแกรมเปิดหน้า `http://127.0.0.1:8000` ให้อัตโนมัติ กด `Ctrl`+`C` ใน Terminal เพื่อปิดโปรแกรม

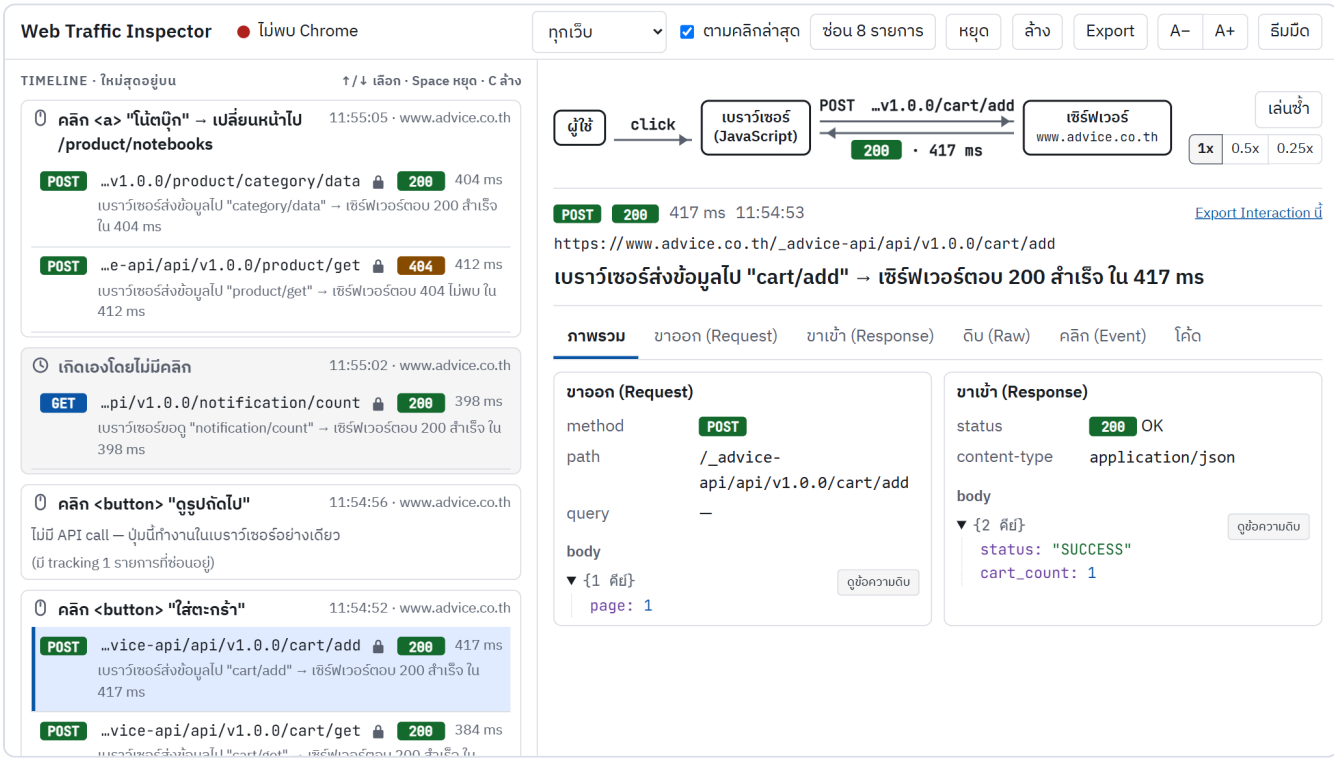
3. **คลิกบนเว็บใน Chrome ตัวนั้น** แล้วดูผลใน Inspector แนะนำให้วางหน้าต่าง Chrome กับ Inspector ซ้าย-ขวาบนจอโปรเจกเตอร์

ถ้ารัน Inspector ก่อนเปิด Chrome หน้าจอจะแสดง "ไม่พบ Chrome ที่เปิดโหมด debug" พร้อมคำสั่งให้กด **คัดลอกคำสั่ง** ไปวางใน Terminal เมื่อ Chrome เปิดขึ้นมา Inspector จะเชื่อมต่อเองภายในไม่กี่วินาที



ภาพที่ 1 · เปิด Inspector ก่อน Chrome: มีคำสั่งเปิด Chrome ให้คัดลอก

4. รู้จักหน้าจอ



ภาพที่ 2 · ซ้าย: Timeline ของแต่ละคลิก · ขวา: Detail ของ Exchange ที่เลือก (ในภาพคือ `cart/add`)

Timeline (คอลัมน์ซ้าย)

- แต่ละก้อนคือหนึ่ง Interaction: **คลิก <button> "ใส่ตะกร้า", เปิดหน้า /** หรือ **เกิดเองโดยไม่มีคลิก** พร้อมเวลาและชื่อเว็บ

- ถ้าคลิกพาไปหน้าใหม่ หัวข้อจะบอกว่า "→ เปลี่ยนหน้าไป ..."
- ในก้อนมีรายการ Exchange: method (`GET` / `POST`), path, status (`200` , `404` ...) และเวลาที่ใช้ พร้อมประโยคสรุปภาษาคน เช่น *เบราว์เซอร์ส่งข้อมูลไป "cart/add" → เซิร์ฟเวอร์ตอบ 200 สำเร็จ ใน 417 ms*
- คลิกที่ไม่มี request เลยจะขึ้นว่า "ไม่มี API call — ปุ่มนี้ทำงานในเบราว์เซอร์อย่างเดียว" ซึ่งเป็นจุดสอนที่ดี

สัญลักษณ์ในแถว Exchange

สัญลักษณ์	ความหมาย
🔒 รูปกุญแจ	มีข้อมูลลับถูกซ่อนไว้ใน Exchange นี้ (แสดงเป็น `••••`)
กรอบ "ไม่ได้ตรวจหาข้อมูลลับ"	body อ่านรูปแบบไม่ออก (เช่น text ธรรมดา, binary, JSON ที่ถูกตัด) จึงตรวจหาค่าลับไม่ได้ **ควรดูก่อนแสดงหน้าห้อง**
"รอคำตอบ..."	ส่ง request ไปแล้ว ยังไม่ได้ response
"failed"	ไม่ได้คำตอบ เช่น เน็ตหลุดหรือถูกยกเลิก
(มี tracking N รายการที่ซ่อนอยู่)	คลิกนี้มี request ไป analytics/tracking ที่ถูกซ่อนไว้ ดูได้ด้วยปุ่ม "ซ่อน N รายการ"

Detail (คอลัมน์ขวา)

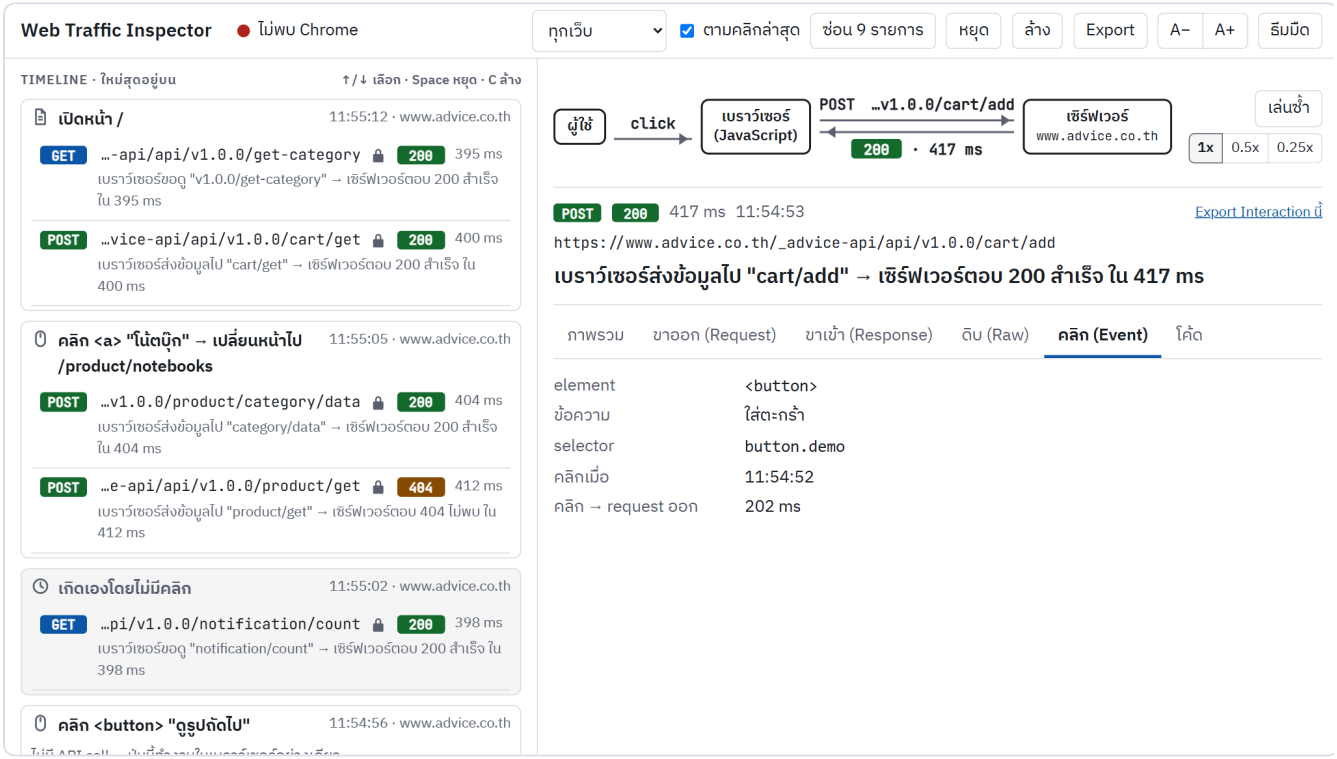
- **Flow strip** ด้านบนเล่นแอนิเมชัน ผู้ใช้ → click → เบราว์เซอร์ → request → เซิร์ฟเวอร์ → response กด **เล่นซ้ำ** เพื่อเล่นอีกครั้ง และเลือกความเร็ว **1x / 0.5x / 0.25x** ให้นักศึกษาตามทัน
- ใต้ Flow strip คือ URL เต็ม, ประโยคสรุป และแท็บรายละเอียด (หัวข้อถัดไป)
- **Export Interaction นี้** ดาวน์โหลดเฉพาะคลิกนี้เป็นไฟล์ Recording

> **ตามคลิกล่าสุด**: เมื่อติ๊กไว้ Detail จะเลือก Exchange หลักของคลิกล่าสุดให้เอง โดยให้ความสำคัญกับ request ที่เป็นการกระทำ (เช่น `cart/add` , `login` , `delete`) มากกว่า request ที่แค่อ่านข้อมูล ถ้าเลือกแถวเองหรือเอาเครื่องหมายออก ระบบจะหยุดเลือกแทน

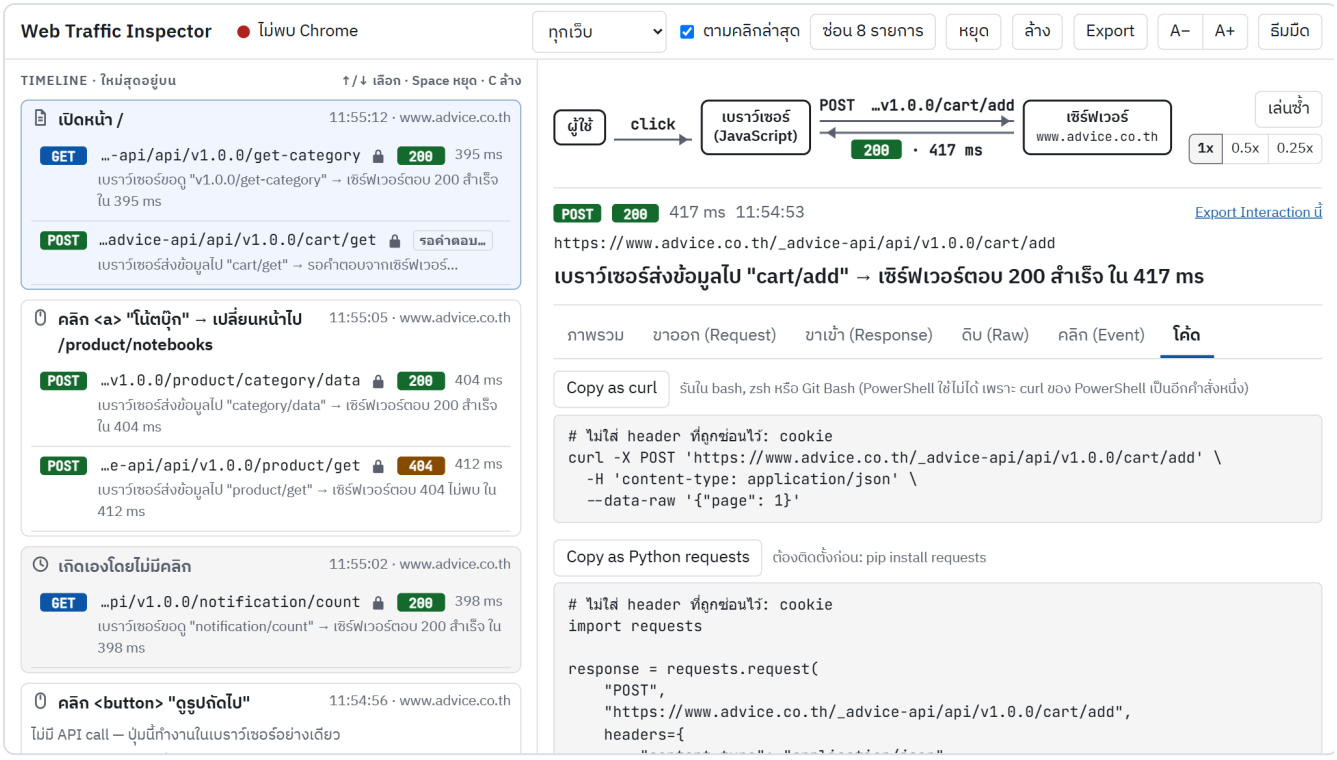
5. แท็บใน Detail

แท็บ	ใช้สอนอะไร
ภาพรวม	สรุปทั้งขาออกและขาเข้าในหน้าเดียว: method, path, query, body ที่ส่ง และ status, content-type, body ที่ได้กลับ
ขาออก (Request)	header, query, cookie และ body ของ request (ค่าลับเป็น `••••`) body ที่เป็น JSON แสดงเป็นต้นไม้ กดพับ/กางได้ และกด "ดูข้อความดิบ" เพื่อดูแบบข้อความ
ขาเข้า (Response)	status, header และ body ของ response
ดิบ (Raw)	request และ response ในรูปแบบข้อความ HTTP ตามที่ส่งจริง (หลังซ่อนค่าลับแล้ว)

คลิก (Event)	element ที่ถูกคลิก, ข้อความบนปุ่ม, CSS selector, เวลาที่คลิก และระยะเวลาจากคลิกถึง request ออก
โค้ด	คัดลอก request นี้เป็น **curl**, **Python requests** หรือ **fetch** เพื่ออธิบายว่าเบราว์เซอร์ทำอะไร header ที่ถูกซ่อน (เช่น cookie) จะไม่ถูกใส่ลงในโค้ด



ภาพที่ 3 · แท็บ คลิก (Event)



ภาพที่ 4 · แท็บ โค้ด

> curl ที่คัดลอกไปใช้ได้ใน bash, zsh หรือ Git Bash แต่ใช้ใน PowerShell ไม่ได้ เพราะ `curl` ของ PowerShell เป็นอีกคำสั่งหนึ่ง

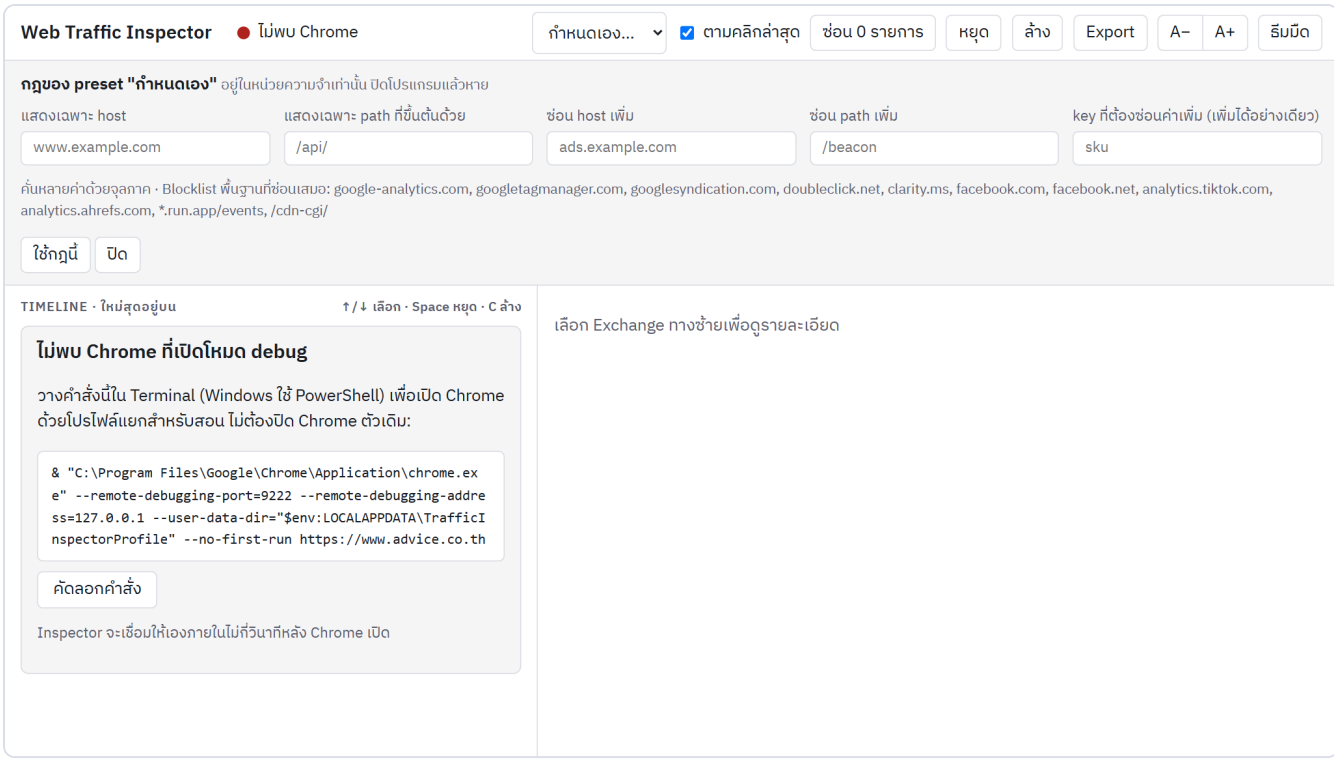
6. แถบเครื่องมือและปุ่มลัด

ปุ่ม	หน้าที่
สถานะ (จุดสี)	เขียว "เชื่อมต่อแล้ว · N แท็บ" / แดง "ไม่พบ Chrome" ถ้า Chrome หลุดจะมีแถบเหลืองแจ้ง และเชื่อมใหม่เอง ข้อมูลเดิมไม่หาย
ช่องเลือก Preset	ทุกเว็บ / advice.co.th / กำหนดเอง... (ดูหัวข้อ 7)
ตามคลิกล่าสุด	ให้ Detail เลือก Exchange หลักของคลิกล่าสุดอัตโนมัติ
ซ่อน N รายการ	สลับแสดง/ไม่แสดง Exchange ที่ถูกซ่อน พร้อมเหตุผล
หยุด / เก็บต่อ	หยุดเก็บชั่วคราวระหว่างอธิบาย (ปุ่มลัด `Space`)
ล้าง	ลบข้อมูลทั้งหมดออกจากหน้าจอและหน่วยความจำ (ปุ่มลัด `C`)
Export	ดาวน์โหลดทุกอย่างที่เก็บไว้เป็นไฟล์ Recording (.json)
A− / A+	ปรับขนาดตัวอักษร (สูงสุด 20 px) ให้คนท้ายห้องอ่านได้
ธีมมืด	สลับธีมสว่าง/มืด
`↑` / `↓`	เลือก Exchange ก่อนหน้า/ถัดไปใน Timeline

7. Preset และ Exchange ที่ถูกซ่อน

เว็บจริงมี request ไป analytics, โฆษณา และ tracking จำนวนมาก Inspector จึงซ่อนสิ่งที่ไม่เกี่ยวข้องให้ เพื่อให้เห็นเฉพาะ API ที่ควรสอน

- **Blocklist พื้นฐาน** ซ่อนเสมอ เช่น google-analytics.com, googletagmanager.com, doubleclick.net, clarity.ms, facebook.com, analytics.tiktok.com
- **ทุกเว็บ** (`all`): แสดงทุกเว็บ ยกเว้น Blocklist พื้นฐาน
- **advice.co.th** (`advice`): แสดงเฉพาะ API ของ advice.co.th ที่เหลือถือว่า "นอกโฟกัส"
- **กำหนดเอง...**: กรอกกฎเองได้ทันทีโดยไม่ต้องแก้ไฟล์ กฎอยู่ในหน่วยความจำเท่านั้น ปิดโปรแกรมแล้วหาย



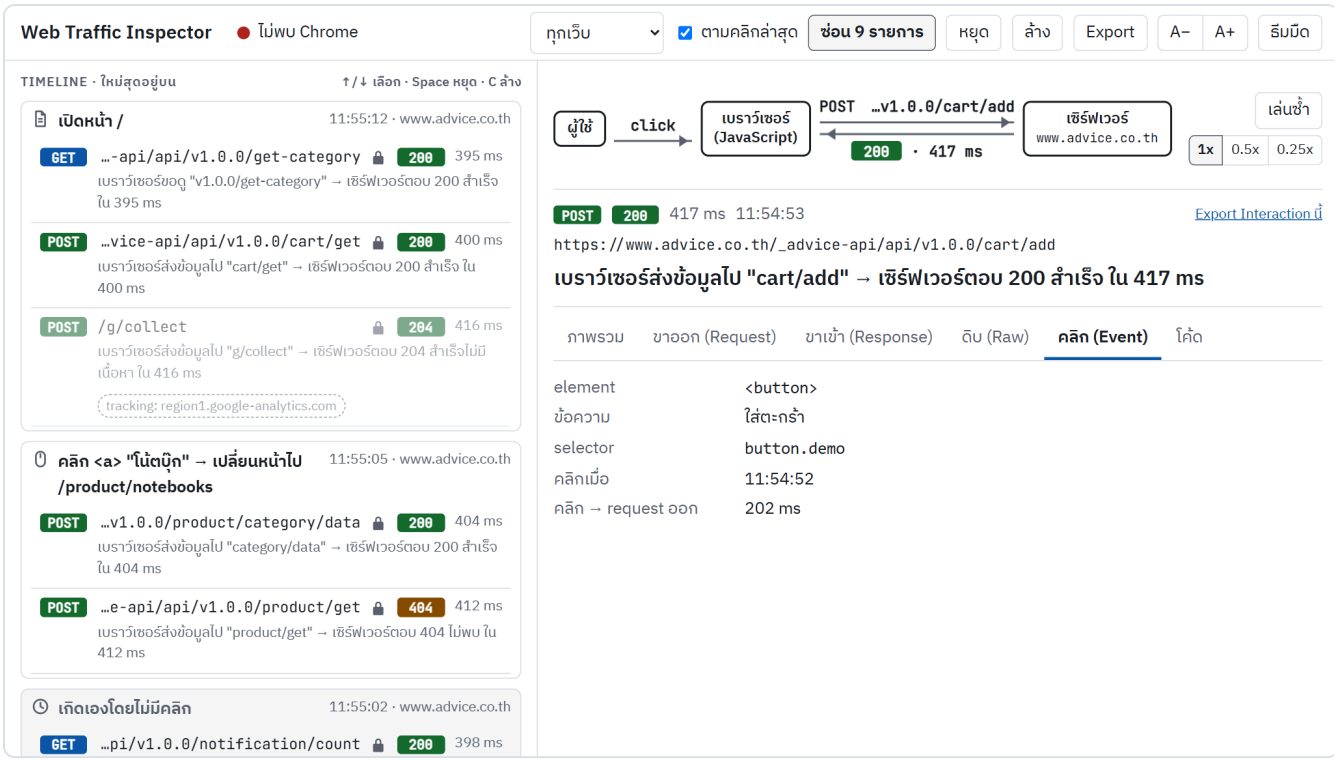
ภาพที่ 5 · ฟอร์ม preset "กำหนดเอง": คั่นหลายค่าด้วยจุลภาค แล้วกด "ใช้กฎนี้"

ช่อง	ตัวอย่าง / ความหมาย
แสดงเฉพาะ host	`www.example.com` แสดงเฉพาะ request ไปเว็บนี้
แสดงเฉพาะ path ที่ขึ้นต้นด้วย	`/api/`
ซ่อน host เพิ่ม / ซ่อน path เพิ่ม	`ads.example.com` , `/beacon`
key ที่ต้องซ่อนค่าเพิ่ม	`sku` ซ่อนค่าของ key นี้เพิ่มจากค่าลับมาตรฐาน (เพิ่มได้อย่างเดียว ลดไม่ได้)

Preset แบบไฟล์อยู่ในโฟลเดอร์ `presets/` (เช่น `advice.json`) ใช้ตอนเริ่มโปรแกรมด้วย `--preset ชื่อไฟล์`

ดูสิ่งที่ถูกซ่อน

กดปุ่ม **ซ่อน N รายการ** แถวที่ถูกซ่อนจะแสดงเป็นสีจางพร้อมป้ายเหตุผล เช่น `tracking: region1.google-analytics.com` หรือ `นอกโฟกัส: preset advice` เหมาะกับการสอนว่าเว็บหนึ่งหน้าส่งข้อมูลไปที่อื่นมากแค่ไหน



ภาพที่ 6 · เปิดดู Exchange ที่ถูกซ่อน: `/g/collect` ไป google-analytics

8. Export และเปิดไฟล์ Recording

- กด **Export** บนแถบเครื่องมือเพื่อดาวน์โหลดทุก Interaction เป็นไฟล์ `.json` หรือกด **Export Interaction นี้** ใน Detail เพื่อเอาเฉพาะคลิกเดียว
- ค่าลับถูกซ่อนตั้งแต่ก่อนเก็บ ไฟล์ Recording จึงไม่มี cookie หรือ token จริง
- เปิดไฟล์ดูย้อนหลังได้โดยไม่ต้องต่อ Chrome และไม่ส่ง request ใดๆ ออกไป:

```
python -m inspector --replay-file รอบซ้อม.json
```

สถานะด้านบนจะขึ้นว่า **Open Recording: <ชื่อไฟล์>** และปุ่มหยุด/ล้างจะถูกซ่อน ถ้าไฟล์มาจาก Inspector รุ่นเก่า ระบบจะซ่อนค่าลับให้อีกรอบตอนเปิด

> ควรเตรียม Recording ของรอบซ้อมไว้เสมอ เผื่อเน็ตในห้องมีปัญหา หรือเว็บเปลี่ยนหน้าตาในวันสอน

9. ตัวเลือก command line

Option	ค่าเริ่มต้น	ความหมาย
`--cdp-url`	`http://127.0.0.1:9222`	ที่อยู่ CDP ของ Chrome ที่จะดู
`--port`	`8000`	พอร์ตของหน้า Inspector (เปิดเฉพาะ 127.0.0.1)
`--preset`	`all`	Preset ตอนเริ่ม เช่น `advice`

`--window-ms`	`1500`	ช่วงเวลาหลังคลิก (ms) ที่นับ Exchange เป็นผลของคลิกนั้น
`--max-exchanges`	`500`	จำนวน Exchange สูงสุดในหน่วยความจำ เกินแล้วตัวเก่าสุดหลุดออก
`--no-open`	ปิด	ไม่เปิดหน้า Inspector ให้อัตโนมัติ
`--demo`	ปิด	ป้อนข้อมูลตัวอย่างแทน Chrome สำหรับลองหน้าจอ
`--replay-file`	ไม่มี	เปิดไฟล์ Recording ดูย้อนหลัง
`--launch-chrome`	ปิด	หา Chrome ในเครื่องแล้วเปิดโหมด debug ให้ (โปรไฟล์แยก)

`--demo` , `--replay-file` และ `--launch-chrome` ใช้พร้อมกันไม่ได้

10. ความปลอดภัยและข้อมูลลับ

> **ใครก็ตามที่เข้าถึงพอร์ต debug 9222 ได้ ควบคุม Chrome ตัวนั้นได้ทั้งหมด** ทั้งอ่าน cookie, เปิดหน้าเว็บ และกดปุ่มแทนเรา

- เปิด Chrome ด้วย `--remote-debugging-address=127.0.0.1` เสมอ (สคริปต์และ `--launch-chrome` ใส่ให้แล้ว) ห้ามเปลี่ยนเป็น `0.0.0.0`
- อย่าเปิดพอร์ตนี้ใน firewall และอย่าใช้ Chrome ตัวนี้ล็อกอินบัญชีส่วนตัว
- ปิด Chrome ตัวนี้เมื่อสอนเสร็จ
- หน้า Inspector เปิดเฉพาะ `127.0.0.1` เครื่องอื่นในแลบเข้าไม่ได้ ถ้าจะให้ทุกคนดูให้ฉายจอแทน

อะไรถูกซ่อนบ้าง

Inspector ซ่อนค่าลับ**ก่อนเก็บ** จึงไม่มีค่าจริงหลุดไปที่หน้าจอ, ไฟล์ Export หรือโค้ดที่คัดลอก ครอบคลุม

- header เช่น `Cookie` , `Set-Cookie` , `Authorization`
- ค่าใน query string, form, JSON และ multipart ที่ชื่อ key บอกว่าเป็นข้อมูลลับ เช่น password, token, secret, session, otp, card, phone, email, address
- รหัสติดตามผู้ใช้ เช่น `_fbp` , `_ga` , `gclid` , `fbclid`
- JSON ที่ส่งเป็น text, JSON แบบคู่ชื่อ/ค่า (`{"name": "_fbp", "value": ...}`) และ JSON ที่ซ้อนอยู่ใน string

body ที่อ่านรูปแบบไม่ออกจะมีป้าย **ไม่ได้ตรวจหาข้อมูลลับ** ให้เปิดดูเองก่อนฉายขึ้นจอ

11. ข้อจำกัดที่ควรรู้

จับคู่คลิกกับ request ด้วยเวลา

Chrome ไม่ได้บอกว่า request ไหนเกิดจากคลิกไหน Inspector จึงนับ Exchange ที่เริ่มภายใน **1500 ms** หลังคลิก (ปรับได้ด้วย `--window-ms`) ในแท็บเดียวกันเป็นผลของคลิกนั้น ถ้าคลิกพาไปหน้าใหม่ ช่วงเวลาขยายต่ออีก 5 วินาทีนับจากตอนเปลี่ยนหน้า ผลคือ

- request เบื้องหลัง (analytics, polling) ที่บังเอิญเกิดในช่วงนั้นจะถูกนับรวมด้วย
- request ที่เริ่มช้ากว่าช่วงเวลาจะไปอยู่ใน "เกิดเองโดยไม่มีคลิก"
- คลิกถี่ๆ ติดกัน request ที่มาช้าของคลิกแรกอาจไปอยู่กับคลิกถัดไป

ตอนสาธิตให้**คลิกทีละครั้งแล้วรอให้ Timeline นิ่ง**ก่อน

อื่นๆ

- ดักเฉพาะ fetch/XHR (การเรียก API) ไม่แสดงรูปภาพ, CSS หรือไฟล์ JavaScript
- การอัปโหลดไฟล์ด้วย FormData: Chrome ไม่ส่ง body ให้ จึงแสดงข้อความแจ้งแทน body
- เปลี่ยน Preset แล้ว การซ่อน/แสดงจะคำนวณใหม่ทันที แต่ "key ที่ต้องซ่อนค่าเพิ่ม" มีผลเฉพาะ Exchange ที่เกิดหลังจากนั้น ถ้าจะเพิ่ม key ให้ตั้งก่อนเริ่มคลิก หรือกด ล้าง แล้วทำใหม่

12. เคล็ดลับการสาธิตในห้อง

ก่อนเข้าห้อง

- ซ้อมเส้นทางที่จะคลิกทั้งหมด แล้ว Export เก็บไว้เป็น Recording สำรอง
- ใช้โปรไฟล์ Chrome แยก (สคริปต์ทำให้แล้ว) ถ้าต้องล็อกอินให้ใช้บัญชีสำหรับสาธิต
- เลือก Preset ให้ตรงกับเว็บ เช่น `--preset advice` เพื่อตัด tracking ออก

ระหว่างสอน

- วาง Chrome กับ Inspector ซ้าย-ขวา แล้วกด **A+** ให้คนท้ายห้องอ่านได้
- คลิกทีละครั้ง รอให้ Timeline นิ่ง แล้วกด `Space` หยุดเก็บระหว่างอธิบาย
- ใช้ Flow strip ความเร็ว 0.5x หรือ 0.25x ตอนอธิบายครั้งแรก
- หยุดก่อนขั้นชำระเงินเสมอ และลบสินค้าออกจากตะกร้าหลังสาธิต (การลบก็เป็น request ที่น่าสอนอีกตัว)

ตัวอย่างลำดับการสอนบน advice.co.th

1. เปิดหน้าแรก: ดู "เปิดหน้า /" ว่าหน้าเว็บเรียก API อะไรบ้างตอนโหลด (เช่น `get-popup-home`, `cart/get`)
2. คลิกเมนูหมวดโน้ตบุ๊ก: ดู `product/category/data` และเปิด response ที่เป็น JSON
3. เข้าหน้าสินค้า → เพิ่มลงตะกร้า: Detail เลือก `cart/add` ให้เอง ดู body ขาออก (รหัสสินค้า, จำนวน) และคำตอบของเซิร์ฟเวอร์
4. เปิดแท็บ ขาออก ชี้ให้เห็นว่า cookie ถูกซ่อนเป็น `••••` และอธิบายว่าทำไมต้องซ่อน

5. กด "ซ่อน N รายการ" ให้ดูว่าเว็บส่งข้อมูลไป google-analytics, clarity ฯลฯ มากแค่ไหน
6. เปิดแท็บ โค้ด ให้ดูว่า request เดียวกันเขียนเป็น curl / Python ได้อย่างไร
7. ลบสินค้าออกจากตะกร้า แล้วดู request การลบ

13. แก้ปัญหาที่พบบ่อย

อาการ	วิธีแก้
ขึ้น "ไม่พบ Chrome" ตลอด	Chrome ที่เปิดอยู่ไม่ได้เปิดโหมด debug ให้เปิดด้วยสคริปต์ใน `scripts/` หรือคำสั่งที่หน้าจอให้คัดลอก หรือรัน `python -m inspector --launch-chrome` (Chrome 136 ขึ้นไปไม่เปิดพอร์ต debug บนโปรไฟล์ปกติ จึงต้องใช้โปรไฟล์แยก)
running scripts is disabled ใน PowerShell	รัน `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` ครั้งเดียว
คลิกแล้วไม่มีอะไรขึ้น	ตรวจว่าคลิกใน Chrome ตัวที่เปิดโหมด debug (โปรไฟล์แยก) ไม่ใช่ Chrome ปกติ, ปุ่ม หยุด ไม่ได้ถูกกดค้างไว้, และ Preset ไม่ได้ซ่อนเว็บนั้น (ลองเลือก "ทุกเว็บ")
ปุ่มนั้นขึ้นว่า "ไม่มี API call"	ปกติ ปุ่มทำงานในเบราว์เซอร์อย่างเดียว เช่น เลื่อนรูปภาพ เปิดเมนู
request ไปอยู่ผิดคลิก	เป็นข้อจำกัดของการจับคู่ด้วยเวลา (หัวข้อ 11) คลิกช้าลง หรือปรับ `--window-ms`
Chrome ปิดไประหว่างสอน	เปิด Chrome ใหม่ด้วยวิธีเดิม Inspector เชื่อมกลับเอง และข้อมูลเดิมยังอยู่
พอร์ต 8000 ถูกใช้อยู่	รันด้วยพอร์ตอื่น เช่น `python -m inspector --port 8080`
อยากลองหน้าจอโดยไม่ต่อเน็ต	`python -m inspector --demo` (สถานะจะขึ้น "ไม่พบ Chrome" เพราะไม่ได้ต่อ Chrome จริง) หรือเปิด Recording ด้วย `--replay-file`

ปิดงานหลังสอน

1. ลบสินค้าที่ใส่ตะกร้าระหว่างสาธิต
2. กด Export ถ้าต้องการเก็บไว้ใช้ครั้งหน้า
3. กด `Ctrl` + `C` ใน Terminal เพื่อปิด Inspector
4. ปิด Chrome ตัวที่เปิดโหมด debug